

User Stories — Digital Logbook Views

1. Calendar View

US-C01: View tasks on a monthly calendar

As a student, **I want** to see my tasks laid out on a month grid by due date, **so that** I can plan my workload across the month at a glance.

Acceptance criteria:

- The calendar displays a 7-column grid (Mon–Sun) for the selected month.
 - Each day cell shows entries whose `due_date` falls on that day.
 - Days with many tasks show the first few entries plus a "+N more" indicator.
 - Only unarchived, non-deleted entries with a due date appear.
-

US-C02: Navigate between months

As a student, **I want** to move forward and backward through months and jump to today, **so that** I can review past deadlines or plan ahead.

Acceptance criteria:

- A "Previous" button moves the view one month back.
 - A "Next" button moves the view one month forward.
 - A "Today" button returns the view to the current month.
 - The month/year label updates to reflect the visible period.
-

US-C03: Switch between month and week views

As a student, **I want** to toggle between a month view and a week view, **so that** I can zoom into a single week when I need more detail.

Acceptance criteria:

- A toggle control switches between Month and Week layouts.
 - The week view shows seven vertical day columns for the selected week.
 - Navigation controls advance or retreat by one week in week mode.
-

US-C04: Reschedule a task by dragging it to another day

As a student, **I want** to drag an entry onto a different day in the calendar, **so that** I can move a deadline without opening the entry editor.

Acceptance criteria:

- Dragging an entry onto another day calls `updateEntry()` with the new `due_date`.
 - The entry appears on the new day immediately after the drop.
 - The server write is persisted; if it fails, the entry reverts to its original day.
-

US-C05: Identify overdue and completed tasks visually

As a student, I want overdue tasks to be highlighted in red and completed tasks to appear in green with a strikethrough, **so that** I can spot urgent work and completed items at a glance.

Acceptance criteria:

- Entries with a `due_date` in the past and a non-done status are rendered with a red visual indicator.
 - Entries with a status of `done_and_dusted` are rendered in green with a strikethrough on the title.
 - Visual indicators are accessible (not colour-only — consider an icon or pattern).
-

2. Kanban Board

US-K01: View tasks grouped by status

As a student, I want to see my tasks arranged in three columns — Up Next, In Motion, and Done & Dusted, **so that** I can understand the state of my work at a glance.

Acceptance criteria:

- Three columns are displayed, one per status.
 - Each card shows the entry title, project name, due date, and priority.
 - Only unarchived, non-deleted entries appear.
-

US-K02: Move a task between columns by dragging

As a student, I want to drag a card from one column to another, **so that** I can update a task's status without opening an editor.

Acceptance criteria:

- Dragging a card to a different column optimistically updates the local state.
 - `updateEntry()` is called with the new status.
 - If the server write fails, the card reverts to its original column.
-

US-K03: Automatic timestamp management on status change

As a student, I want the system to automatically set `started_at` when I move a task to In Motion and `ended_at` when I move it to Done & Dusted, **so that** I don't have to track time manually.

Acceptance criteria:

- Moving a card to "In Motion" sets `started_at` to the current timestamp (if not already set).
 - Moving a card to "Done & Dusted" sets `ended_at` to the current timestamp.
 - Moving a card back to "Up Next" does not clear existing timestamps.
-

US-K04: Filter the board by project or search term

As a student, I want to filter the Kanban board by project or type a search query, **so that** I can focus on a specific project or find a task quickly.

Acceptance criteria:

- A project dropdown narrows the board to entries belonging to the selected project.
- A search input filters cards whose title or description matches the query.
- Filters apply in real time without a page reload.

- Clearing the filter restores the full board.
-

US-K05: Identify overdue tasks on the board

As a student, **I want** overdue cards to be visually highlighted, **so that** I can prioritise them while working on the board.

Acceptance criteria:

- Cards with a `due_date` in the past and a non-done status are rendered with a red visual indicator.
 - The highlight is visible in both light and dark themes.
-

3. Timeline

US-T01: View tasks as horizontal bars across time

As a student, **I want** to see my tasks as bars spanning their start date to their due date on a horizontal timeline, **so that** I can visualise how my work is distributed over time.

Acceptance criteria:

- Each entry is rendered as a bar from its resolved start date to its resolved end date.
 - Start date resolution: `started_at` → `created_at` → one day before `due_date`.
 - End date resolution: `due_date` → one day after the resolved start.
 - Entries that cannot be placed on a date are skipped.
 - A red dashed "today" line marks the current date.
 - Grid lines and month-start markers provide temporal context.
-

US-T02: See dependency arrows between linked tasks

As a student, **I want** to see curved arrows drawn from a predecessor task to its successor, **so that** I can understand which tasks depend on others and plan accordingly.

Acceptance criteria:

- Dependencies are read from `entries.dependencies` or `entries.depends_on` in the JSONB payload.
 - A Bézier arrow is drawn from the predecessor's right edge to the successor's left edge.
 - A three-task chain (A → B → C) appears as three connected bars with arrows, not as three unrelated bars.
 - Tasks in a chain are placed on separate rows to keep arrows legible.
-

US-T03: Zoom in and out on the timeline

As a student, **I want** to zoom the timeline from 50% to 400%, **so that** I can see fine detail for a busy week or the big picture across a month.

Acceptance criteria:

- Zoom controls offer at least 0.5x, 1x, 2x, 3x, and 4x levels.
 - At higher zoom levels, individual day labels become visible.
 - The view remains horizontally scrollable at all zoom levels.
-

US-T04: Scroll horizontally across the timeline

As a student, I want to scroll left and right across the timeline, **so that** I can navigate to tasks in the past or future without changing zoom.

Acceptance criteria:

- The timeline container supports horizontal scrolling.
 - Scrolling is smooth and does not break the grid or bar rendering.
-

US-T05: See a helpful empty state when no dated tasks exist

As a new student, I want to see a friendly message when the timeline has nothing to show, **so that** I understand what I need to do to populate it.

Acceptance criteria:

- When no unarchived entries have a placable date, an empty state is shown.
 - The empty state explains how to add tasks with start/due dates and how to set dependencies.
-

4. Import & Export (Data Portability)

US-D01: Export all data as JSON

As a student, I want to download all my projects and entries as a JSON file, **so that** I have a machine-readable backup of my entire logbook.

Acceptance criteria:

- The export includes all projects (active and archived) and all entries (active and archived).
 - The JSON file contains a version field, export timestamp, user email, projects array, and entries array.
 - The file downloads automatically when the button is clicked.
 - The export is round-trip safe: importing it into an empty database reproduces the original row count.
-

US-D02: Export all data as CSV

As a student, I want to download all my projects and entries as a CSV file, **so that** I can open the data in a spreadsheet for analysis or reporting.

Acceptance criteria:

- Projects and entries are emitted as separate sections, each with a header row.
 - The `entries` JSONB field is serialised as a JSON string within the cell.
 - Commas, quotes, and newlines within values are properly escaped.
 - The file opens correctly in Excel, Google Sheets, and LibreOffice Calc.
-

US-D03: Export all data as Markdown

As a student, I want to download all my projects and entries as a Markdown file, **so that** I can paste the data into documentation, a wiki, or a README.

Acceptance criteria:

- The Markdown file contains a metadata header (user, date, counts), a Projects table, and an Entries table.
 - Pipe characters within values are escaped to preserve table structure.
 - The file renders correctly on GitHub and in standard Markdown viewers.
-

US-D04: Import data from a previously exported file

As a student, **I want** to upload a JSON, CSV, or Markdown export file and have it recreate my projects and entries, **so that** I can restore a backup or migrate to a new account.

Acceptance criteria:

- The format is auto-detected from the file extension (`.json` , `.csv` , `.md`).
 - Projects are created first, then entries, then archived entries are re-archived.
 - An import report shows the count of projects created, entries created, and rows rejected.
 - The imported data appears in the Dashboard, Calendar, Kanban, Timeline, and Today views without a page refresh.
-

US-D05: Receive a report of rejected rows during import

As a student, **I want** the import to report which rows it rejected and why (with line numbers), **so that** I can fix malformed data and re-import without losing valid rows.

Acceptance criteria:

- Invalid rows are skipped, not treated as a hard failure.
 - Each rejection reports a 1-based line number and a reason (e.g., "Missing or empty project_name", "Invalid status: bogus").
 - Valid rows are still imported even when some rows are rejected.
 - The rejection list is displayed in the import report after the operation completes.
-

US-D06: Drag and drop a file to import

As a student, **I want** to drag a file onto the import area instead of clicking through a file picker, **so that** importing feels fast and natural.

Acceptance criteria:

- The import area highlights when a file is dragged over it.
 - Dropping a file triggers the same parse-and-import flow as the file picker.
 - The file picker also works via a standard click on the label.
-

US-D07: Round-trip safety

As a student, **I want** an export-then-import cycle to reproduce my data exactly, **so that** I can trust the backup and restore process.

Acceptance criteria:

- Exporting and then importing into an empty database produces the same number of project rows and entry rows.
 - Archived projects remain archived after import.
 - Archived entries are re-archived after import.
 - All entry fields (title, due date, priority, status, timestamps, JSONB payload) are preserved.
-

US-D08: Export to iCalendar (.ics)

As a student, **I want** to export my tasks as an iCalendar (.ics) file, **so that** I can open or subscribe to them in Google Calendar, Outlook, or Apple Calendar.

Acceptance criteria:

- The .ics file is RFC 5545 compliant and opens correctly in major calendar applications.
 - Entries with only a `due_date` are exported as all-day events (`DTSTART;VALUE=DATE`).
 - Entries with a `started_at` timestamp are exported as timed events (`DTSTART` with time).
 - The entry title is mapped to `SUMMARY` , project name to `CATEGORIES` , and status to `STATUS` (TENTATIVE/CONFIRMED/COMPLETED).
 - Priority is mapped to the iCalendar `PRIORITY` field (1–9 scale).
 - Special characters (semicolons, commas, newlines, backslashes) are escaped per RFC 5545.
 - Entries without any date are skipped and do not appear in the calendar.
-

Feature 5: Backup, Restore & Migrations

US-B01: One-command database backup

As a student, **I want** to back up my entire database with a single command, **so that** I can recover my data if something goes wrong.

Acceptance criteria:

- Running `npm run db:backup` creates a PostgreSQL custom-format dump file.
- The backup includes all app tables (users, projects, entries, fields, activity_log, health_ping, ai_provider_cooldowns).
- The backup file is saved with a timestamp in `scripts/backups/` .
- A custom path can be provided: `npm run db:backup -- ./my-backup.dump` .

US-B02: One-command database restore

As a student, **I want** to restore my database from a backup with a single command, **so that** I can recover after data loss.

Acceptance criteria:

- Running `npm run db:restore` restores from the most recent backup in `scripts/backups/` .
- A specific file can be provided: `npm run db:restore -- ./my-backup.dump` .
- The restore shows a 3-second warning before overwriting data.
- After restore, running `npm run db:migrate` brings the schema up to date.

US-B03: Versioned schema migrations

As a developer, **I want** to run versioned schema migrations that upgrade an existing database, **so that** I never have to drop and recreate tables when the schema changes.

Acceptance criteria:

- `npm run db:migrate` runs all pending migrations in order.
- Each migration is tracked in a `schema_migrations` table with version and checksum.
- Migrations already applied are skipped on subsequent runs.
- Failed migrations stop execution and leave the database in a consistent state (transactional).
- `npm run db:status` shows which migrations are applied and pending.

US-B04: Bootstrap existing database

As a developer with a database created before the migration system, **I want** to mark existing migrations as already applied, **so that** the migration runner doesn't try to re-run them.

Acceptance criteria:

- `npm run db:bootstrap` marks all migration files as applied without executing them.
- After bootstrap, `npm run db:migrate` only runs NEW migrations going forward.
- A database made before recent schema changes still opens under the current build with data intact.

US-B05: Baseline full schema migration

As a developer setting up a fresh Supabase project, **I want** a single baseline migration that creates the entire current schema, **so that** I don't have to run 8 separate migration files manually.

Acceptance criteria:

- `000_baseline_full_schema.sql` creates all tables, types, indexes, functions, triggers, and cron jobs.
- Every statement is idempotent (`IF NOT EXISTS` , `IF NOT EXISTS` guards).
- Running it against an existing database is a no-op.
- Running it against a fresh database produces a fully working schema.

Feature 6: OpenAPI 3 & Swagger UI

US-A01: Browse the API documentation

As a developer, **I want** to open a browsable page that shows all API endpoints, **so that** I can understand the API without reading source code.

Acceptance criteria:

- Navigating to `/api-docs` on the project service shows a Swagger UI page.
- The page lists all endpoints across all four microservices (project, dashboard, profile, auth).
- Each endpoint shows its HTTP method, path, request body, and response schemas.

US-A02: Try an API endpoint from the docs page

As a developer, **I want** to execute an API call directly from the docs page, **so that** I can verify the API works without switching to Postman.

Acceptance criteria:

- The "Authorize" button accepts a Bearer JWT token.
- The "Try it out" button on any endpoint sends a real request to the running service.
- The response body, status code, and headers are displayed in the docs page.
- A request made from the docs page against the running application succeeds.

US-A03: Spec matches the implemented routes

As a developer, **I want** the OpenAPI spec to match the implemented routes exactly, **so that** the docs page is a reliable source of truth.

Acceptance criteria:

- The spec has no path that the code does not implement.
- The code has no route that the spec does not document.
- Status codes and error bodies in the spec match what the code returns.
- 12 automated tests verify spec structure, paths, and response schemas.